Experiment 1: Informed Search Algorithms

Q1. Implement A* Search for Route Finding

Problem statement: Develop a Python program to find the shortest path between two cities using the A* search algorithm.

Tasks to be performed:

1. Represent at least 8 cities as a weighted graph.
1. Assign edge costs representing distances and heuristic values representing estimated distances to the destination.
2. Implement A* using f(n)=g(n)+h(n)
3. Accept the source and destination cities from the user.
4. Display the selected route, total path cost and nodes explored.

Q2. Implement Greedy Best-First Search

Problem statement: Develop a route-finding system using Greedy Best-First Search.

Tasks to be performed:

1. Create a weighted graph containing at least 8 locations.
2. Assign heuristic values to the locations.
3. Implement Greedy Best-First Search using the heuristic (h(n).
4. Find a route between user-selected source and destination nodes.
5. Display the route, total path cost and number of nodes explored.

Q3. Implement Robot Navigation Using A* Search

Problem statement: Develop a grid-based navigation system to find a path for a robot moving from a starting point to a goal while avoiding obstacles.

Tasks to be performed:

1. Create a grid of at least 8 × 8 cells.
2. Mark the starting cell, goal cell and obstacle cells.
3. Implement A* Search using Manhattan distance as the heuristic.
4. Display the shortest valid path on the grid.
5. Calculate the path length and number of explored cells.

Experiment 2: Constraint Satisfaction Problems (CSP)

Q4. Solve the N-Queens Problem Using CSP

Problem statement: Implement a CSP-based solution to place N queens on an N × N chessboard without conflicts.

Tasks to be performed:

1. Accept the value of N from the user, with N at least 4.
2. Define the variables, domains and constraints.
3. Implement backtracking with constraint checking.
4. Display one valid board configuration.
5. Verify that no two queens share the same row, column or diagonal.
6. Report whether a solution was found.

Q5. Develop a Sudoku Solver Using CSP

Problem statement: Create a Python program to solve a partially filled 9 × 9 Sudoku puzzle.

Tasks to be performed:

1. Accept a Sudoku grid containing empty cells.
2. Define the domain and row, column and 3 × 3 subgrid constraints.
3. Implement a backtracking algorithm to assign valid values.
4. Display the completed Sudoku grid.
5. Validate the final solution against all constraints.
6. Detect and report an unsolvable puzzle.

Q6. Implement Examination Timetable Scheduling Using CSP

Problem statement: Develop a timetable generator that assigns examinations to time slots without scheduling conflicting examinations simultaneously.

Tasks to be performed:

1. Define at least 6 examinations, 4 time slots and a set of student enrollment records.
2. Identify pairs of examinations that cannot occur simultaneously.
3. Define examination variables, available time slots and conflict constraints.
2. Implement backtracking to generate a valid timetable.
4. Display the examination schedule and verify that no conflicts exist.

5. Report if the available time slots are insufficient for a valid schedule.

Experiment 3: Game Playing and Puzzle Solving

Q7. Implement Tic-Tac-Toe Using Minimax

Problem statement: Develop a Tic-Tac-Toe game in which a human plays against a computer using the Minimax algorithm.

Tasks to be performed:

1. Create a 3 × 3 game board.
2. Accept valid moves from the human player.
3. Implement Minimax to select the computer's move.
4. Check for winning, losing and draw conditions.
5. Display the board after each move.
6. Demonstrate that the computer avoids moves that lead to a preventable loss.

Q8. Solve the 8-Puzzle Using A* Search

Problem statement: Implement A* Search to transform a given 8-puzzle configuration into a goal configuration.

Tasks to be performed:

1. Define the initial and goal states.
2. Generate valid moves of the empty tile.
3. Use Manhattan distance as the heuristic.
4. Implement A* Search with repeated-state detection.
5. Display the sequence of puzzle states from start to goal.
6. Report the number of moves and states explored.

Experiment 4: EDA, PCA and Regression

Q9. Uber Fare Prediction Using PCA and EDA

Problem statement: Analyze Uber fare data and investigate whether PCA can reduce dimensionality while maintaining acceptable prediction performance.

Dataset: Uber Fares Dataset — Kaggle

Tasks to be performed:

3. Load the dataset and inspect its dimensions, data types, missing values and summary statistics.
4. Perform EDA using appropriate plots and identify outliers or invalid records.
5. Prepare relevant features and the fare amount as the target variable.
6. Split the data into training and testing sets, and standardize the features where required.
7. Apply PCA and select components using explained variance.
8. Train comparable regression models with and without PCA.
9. Compare R^2, RMSE and MAE, and interpret the results.

Q10. House Price Prediction Using Linear Regression

Problem statement: Develop a regression model to estimate house prices based on property characteristics.

Dataset: House Price — Kaggle

Tasks to be performed:

1. Load a suitable housing dataset.
2. Perform EDA and identify the input features and target variable.
3. Handle missing values and encode categorical features if present.
4. Split the data into training and testing sets.
5. Train a Linear Regression model.
6. Predict house prices for the test set.
7. Evaluate the model using R^2, RMSE and MAE, and plot actual versus predicted prices.

Q11. Student Performance Prediction Using Regression

Problem statement: Predict students' final examination scores using academic and behavioral attributes.

Dataset: student performance — Kaggle

Tasks to be performed:

1. Load a student performance dataset.
2. Examine relationships between study hours, attendance, previous marks and final scores.
3. Select suitable input features and the numerical target variable.
4. Train a Linear Regression model using a train-test split.
5. Predict scores for unseen student records.
6. Evaluate using R^2, RMSE and MAE.

7. Identify the features associated with higher or lower predicted performance.

Q12. Salary Estimation Using Linear Regression

Problem statement: Build a model to estimate employee salary based on years of experience.

Tasks to be performed:

1. Load or create a suitable salary dataset.
2. Visualize the relationship between experience and salary.
3. Split the data into training and testing sets.
4. Train a Linear Regression model.
5. Display the regression equation and fitted regression line.
6. Predict salary for a user-entered number of years of experience.
7. Evaluate the model using R^2, RMSE and MAE.

Q13. Compare Regression Models With and Without PCA

Problem statement: Investigate the effect of dimensionality reduction on the performance of a regression model.

Tasks to be performed:

1. Select a numerical regression dataset containing multiple input features.
2. Preprocess the data and create training and testing sets.
3. Train a regression model using the original features.
4. Apply standardization and PCA using only the training data to avoid data leakage.
5. Train the same regression model on the PCA-transformed features.
6. Evaluate both models using R^2, RMSE and MAE on the test data.
7. Compare predictive performance, number of features and computational implications.

Experiment 5: Decision Tree, KNN and Naïve Bayes

Q14. Loan Approval Prediction Using Decision Tree

Problem statement: Predict whether a loan application should be approved using applicant information.

Tasks to be performed:

1. Load a labeled loan approval dataset.
2. Identify the target variable and relevant applicant features.
3. Handle missing values and encode categorical variables.
4. Split the dataset into training and testing sets.
5. Train a Decision Tree classifier using Gini impurity or entropy.
6. Visualize the tree and predict approval for test records.
7. Display accuracy, precision, recall, F1-score and the confusion matrix.

Q15. Customer Churn Prediction Using Decision Tree

Problem statement: Classify customers according to whether they are likely to discontinue a service.

Tasks to be performed:

1. Load a labeled customer churn dataset.
2. Perform EDA and inspect class distribution.
3. Prepare the features and target variable.
4. Train a Decision Tree classifier.
5. Predict churn for the test dataset.
6. Display the confusion matrix, precision, recall and F1-score.

Q16. Disease Classification Using Decision Tree

Problem statement: Develop a Decision Tree classifier using a suitable labeled public medical dataset.

Tasks to be performed:

1. Load the dataset and identify the diagnostic target.

2. Inspect missing values, feature distributions and class imbalance.
3. Split the dataset into training and testing sets.
4. Train a Decision Tree classifier.
5. Predict the classes of unseen test records.
6. Display accuracy, precision, recall, F1-score and the confusion matrix.

Q17. Decision Tree Optimization Using Entropy and Gini

Problem statement: Compare Decision Tree models built using different splitting criteria and hyperparameters.

Tasks to be performed:

1. Load a suitable classification dataset.
2. Prepare the data and use a fixed train-test split.
3. Train one Decision Tree using Gini impurity and another using entropy.
4. Compare the tree structures and test-set performance.
5. Tune maximum depth and minimum samples per leaf using cross-validation.

Q18. Iris Flower Classification Using KNN

Problem statement: Classify Iris flowers into their species using the K-Nearest Neighbors algorithm.

Tasks to be performed:

1. Load the Iris dataset.
2. Separate the four measurement features and the species label.
3. Split the data into training and testing sets.
4. Standardize the features using training-set statistics.
5. Train KNN classifiers using at least three different K values.
6. Compare test accuracy and select a suitable K.

Q19. Student Performance Classification Using KNN

Problem statement: Classify students into predefined academic performance categories using KNN.

Tasks to be performed:

1. Load a labeled student performance dataset.
2. Select relevant features, such as attendance, study hours and internal marks.
3. Define or use existing performance categories as the target.
4. Split the dataset and scale the input features.
5. Implement KNN and test different K values.
6. Predict categories for unseen student records.
7. Report accuracy, precision, recall and the confusion matrix.

Q20. Diabetes Classification Using KNN

Problem statement: Use KNN to classify records in a suitable public diabetes dataset.

Tasks to be performed:

1. Load the dataset and identify the target class.
2. Inspect missing values, invalid zero values where applicable and feature distributions.
3. Split the data using stratification where appropriate.
4. Standardize the features using a training-fitted scaler.
5. Train KNN models with different K values.
6. Compare model performance using precision, recall, F1-score and confusion matrix.

Q21. Email or SMS Spam Detection Using Naïve Bayes

Problem statement: Develop a text classifier that distinguishes spam messages from legitimate messages.

Tasks to be performed:

1. Load a labeled spam message dataset.
2. Clean the text and separate the message content from its class label.
3. Convert text into numerical features using CountVectorizer or TF-IDF.
4. Split the dataset into training and testing sets without fitting the vectorizer on test data.

5. Train a Multinomial Naïve Bayes classifier.
6. Predict spam and non-spam labels for test messages.
7. Display precision, recall, F1-score and the confusion matrix.

Q22. Iris Classification Using Gaussian Naïve Bayes

Problem statement: Apply Gaussian Naïve Bayes to classify Iris flowers and compare its performance with a Decision Tree.

Tasks to be performed:

1. Load the Iris dataset and identify features and labels.
2. Split the dataset into training and testing sets.
3. Train a Gaussian Naïve Bayes classifier.
4. Train a Decision Tree classifier using the same split.
5. Evaluate both classifiers using accuracy, precision, recall and F1-score.
6. Display the confusion matrices and compare classification performance.
7. Explain the basic independence assumption of Naïve Bayes.

Experiment 6: Ensemble Learning and K-Means Clustering

Q23. Compare Random Forest and Decision Tree

Problem statement: Evaluate the effect of ensemble learning by comparing Random Forest with a single Decision Tree classifier.

Tasks to be performed:

1. Load a suitable labeled dataset, such as Iris or Titanic.
2. Preprocess the data and create a common train-test split.
3. Train a Decision Tree classifier.
4. Train a Random Forest classifier on the same training data.
5. Evaluate model performance -precision, recall and F1-score.

Q24. Titanic Survival Prediction Using Random Forest

Problem statement: Predict passenger survival using a Random Forest classifier.

Tasks to be performed:

1. Load the Titanic dataset.
2. Select relevant features such as passenger class, age, fare and sex.
3. Handle missing values and encode categorical variables.
4. Split the dataset into training and testing sets.
5. Train a Random Forest classifier.
6. Evaluate the predictions using accuracy, precision, recall, F1-score and a confusion matrix.

Q25. Clustering Using K-Means

Problem statement: Segment/cluster customers using their annual income and spending score.

Tasks to be performed:

1. Load a customer dataset containing annual income and spending score.
2. Inspect the feature distributions and missing values.
3. Scale the selected features where appropriate.
4. Apply the elbow method to evaluate candidate values of K.
5. Fit K-Means using a suitable number of clusters.
6. Plot the customer clusters and centroids.
7. Describe the characteristics of each segment.

Q26. Student Grouping Using K-Means

Problem statement: Group students into clusters according to their academic performance.

Tasks to be performed:

1. Load a numerical dataset containing student academic attributes.
2. Select at least three relevant features, such as attendance, internal marks and assignment scores.
3. Standardize the selected features.
4. Apply the elbow method to select a suitable cluster count.

5. Train and Visualize the clusters using a suitable two-dimensional projection.
6. Interpret the cluster profiles without treating the clusters as definitive measures of ability.

Q27. Mall Customer clustering/classification

Problem statement: Identify distinct customer groups using the Mall Customers dataset.

Tasks to be performed:

1. Load the dataset containing CustomerID, Gender, Age, Annual Income (k$) and Spending Score (1-100).
2. Select annual income and spending score for clustering.
3. Standardize the features and use the elbow method to select K.
4. Fit the K-Means algorithm.
5. Plot the clusters with their centroids.
6. Calculate the number of customers in each cluster.
7. Visualize the clusters

Q28. Evaluate K-Means Clustering

Problem statement: Analyze how the number of clusters affects K-Means performance on a numerical dataset.

Tasks to be performed:

1. Select a numerical dataset with at least three features.
2. Handle missing values and standardize the features.
3. Fit K-Means models for at least four different values of K.
4. Select a suitable cluster count using the evaluation results (elbow method)
5. Visualize and interpret the final clusters.

Experiment 7: Apriori Association Rule Mining

Q29. Market Basket Analysis Using Apriori

Problem statement: Discover frequent itemsets and association rules from retail transactions using Apriori.

Tasks to be performed:

1. Load a transactional dataset containing at least 10 transactions.

2. Convert the transactions into a suitable one-hot encoded representation.
3. Apply Apriori with a specified minimum support threshold.
4. Generate association rules using a specified minimum confidence threshold.
5. Calculate and display support, confidence and lift.

Experiment 8: Artificial Neural Networks

Q30. Bank Customer Churn Prediction Using ANN

Problem statement: Develop a neural network classifier to predict whether a bank customer will leave within the period represented by the dataset.

Dataset: Bank Customer Churn Modeling — Kaggle

Tasks to be performed:

1. Load the dataset and inspect its features and target variable.
2. Separate the input features from the churn label.
3. Encode categorical variables and scale numerical features using training data.
4. Divide the dataset into training and testing sets.
5. Build an ANN using an input layer, at least one hidden layer and an output layer.
6. Compile and train the model using suitable loss and optimization functions.
7. Predict churn labels and display accuracy and the confusion matrix.

Q31. Student Pass/Fail Prediction Using ANN

Problem statement: Build an Artificial Neural Network to predict whether a student will pass or fail based on academic attributes.

Tasks to be performed:

1. Load a labeled student performance dataset with a pass/fail target.
2. Select suitable input features and encode categorical data where necessary.
3. Split the data into training and testing sets.
4. Normalize numerical features using training-set statistics.
5. Create and train a feed-forward neural network for binary classification.
6. Predict test-set labels and display accuracy, precision, recall and a confusion matrix.

Experiment 9: Integrated Classification and ML Pipeline

Q32. Comparative Study of Decision Tree, KNN and Naïve Bayes

Problem statement: Compare three classification algorithms on the same real-world dataset.

Tasks to be performed:

1. Select a suitable labeled dataset with at least two classes.
2. Handle missing values and encode categorical features.
3. Create a common stratified train-test split.
4. Train Decision Tree, KNN and Naïve Bayes classifiers.
5. Apply feature scaling for KNN where appropriate.
6. Compare test accuracy, precision, recall, F1-score and confusion matrices.
7. Present a comparison table and recommend a model based on the results.

Q33. Develop an End-to-End Machine Learning Pipeline

Problem statement: Develop a complete machine learning pipeline for a real-world classification or regression problem of the student's choice.

Tasks to be performed:

1. Select a suitable dataset and define the prediction objective.
2. Perform EDA and identify the target variable, feature types and data quality issues.
3. Preprocess the data using suitable imputation, encoding and scaling methods.
4. Split the dataset into training and testing sets without data leakage.
5. Train an appropriate ML algorithm and justify the choice.
6. Tune at least one hyperparameter using cross-validation.
7. Evaluate the final model using suitable classification or regression metrics.